



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ
ΥΠΟΛΟΓΙΣΤΩΝ

Κάργας Στέφανος Διονύσης Κατσέτης
(el23059) (el23005)

Λειτουργικά Συστήματα Υπολογιστών
6ο εξάμηνο, Ακαδημαϊκή περίοδος 2025-2026

2η Εργαστηριακή Άσκηση:
Συγχρονισμός

Αθήνα 2026

Μέρος 1

Κατά την εκτέλεση του προγράμματος μέσω του παρεχόμενου Makefile, παρατηρείται ότι η τελική τιμή της κοινόχρηστης μεταβλητής *val* δεν είναι μηδέν, αλλά ακολουθεί μια στοχαστική εξέλιξη, όντας αποτέλεσμα που αλλάζει σε κάθε εκτέλεση.

- Το Makefile περνάει αυτόματα τις σημαίες μεταγλώττισης (όπως το `-DSYNC_MUTEX` και `-DSYNC_ATOMIC`) ως μακροεντολές `#define` στον προεπεξεργαστή της C και παράγει επιτυχώς τα δύο διαφορετικά εκτελέσιμα από το ίδιο πηγαίο αρχείο. Ωστόσο, επειδή δεν έχουν προστεθεί ακόμα οι κώδικες για τα mutexes ή τα atomics, ο κώδικας παραμένει επί της ουσίας ασυγχρόνιστος και το αποτέλεσμα είναι, όπως αναμενόταν, διάφορο του μηδενός.

Αυτό συμβαίνει διότι η **αύξηση και η μείωση μιας μεταβλητής δεν είναι ατομικές λειτουργίες.**

- Μεταφράζονται σε τρεις ξεχωριστές εντολές Assembly: φόρτωμα από τη μνήμη στους καταχωρητές (Load), αριθμητική πράξη (Add/Sub), και εγγραφή της νέας τιμής πίσω στη μνήμη (Store). Επειδή τα δύο νήματα εκτελούνται ταυτόχρονα, το Λειτουργικό Σύστημα μπορεί να προκαλέσει εναλλαγές περιβάλλοντος λειτουργίας ενδιάμεσα σε αυτά τα βήματα, και να “μπερδευτούν” αυτές οι 3 εντολές. Έτσι δημιουργείται Race Condition και το αποτέλεσμα του μετρητή γίνεται απρόβλεπτο.

Ερώτημα 1

- Ασυγχρόνιστο `simplesync.c`:

```
[oslab018@os-node2:~/sync$ time ./simplesync-mutex
About to decrease variable 10000000 times
About to increase variable 10000000 times
Done increasing variable.
Done decreasing variable.
NOT OK, val = -9383592.

real    0m0.563s
user    0m1.096s
sys     0m0.004s
```

Εκτελώντας το ασυγχρόνιστο `simplesync-mutex` με την εντολή *time* μπροστά από το path του εκτελέσιμου (που ουσιαστικά είναι το ίδιο με το

ασυγχρόνιστο `sync-atomic`, καθώς δεν έχουμε προσθέσει κώδικα συγχρονισμού όπου απαιτείται, απλά ονομάστηκαν έτσι λόγω της δομής του παρεχόμενου `Makefile`) καταλήξαμε στο εξής αποτέλεσμα:

Παρατηρούμε, αρχικά, ότι ο χρόνος χρήστη (`user 1.096s`) είναι σχεδόν διπλάσιος του συνολικού πραγματικού χρόνου (`real 0.563s`). Γεγονός που αποδεικνύει ότι το Λειτουργικό Σύστημα χρονοδρομολόγησε τα δύο νήματα σε διαφορετικούς πυρήνες. Επιτεύχθηκε δηλαδή πλήρης παράλληλη εκτέλεση.

Έπειτα, `sys 0.004s`. Ο κώδικας πέρασε ελάχιστο, πρακτικά μηδενικό χρόνο μέσα σε `kernel mode`. Δεν υπήρξε καμία χρονοβόρα κλήση συστήματος (`system call`) για συγχρονισμό ή διακοπή. Το απειροελάχιστο `0.004s` δικαιολογείται αποκλειστικά από βασικές λειτουργίες, όπως η αρχική δημιουργία των νημάτων και η χρήση της `printf`.

Τέλος, `real 0.563s`. Ο χρόνος ολοκλήρωσης ήταν εξαιρετικά γρήγορος, ακριβώς επειδή τα νήματα δεν συγχρονίστηκαν ποτέ.

➤ Συγχρονισμένο `sync.c` με `mutexes` (εκτελέσιμο `sync-mutex`):

```
oslab018@os-node2:~/sync$ time ./sync-mutex
About to increase variable 10000000 times
About to decrease variable 10000000 times
Done decreasing variable.
Done increasing variable.
OK, val = 0.

real    0m4.199s
user    0m5.058s
sys     0m2.952s
```

Το αποτέλεσμα του `val` είναι σωστό και ντετερμινιστικό σε κάθε εκτέλεση. Παρατηρούμε ότι ο μετρητής παίρνει πάντα τη τιμή μηδέν που

είναι και η ορθή. Αυτό είναι αναμενόμενο, αφού συγχρονίσαμε τα νήματα. Πριν κάθε ένα νήμα μπει στο κρίσιμο τμήμα του (αφαίρεση και πρόσθεση μετρητή **ip* μέσα σε κάθε λούπα) έκανε αίτημα να αποκτήσει και να κλειδώσει ένα mutex. Αν αυτό δεν ήταν διαθέσιμο (κατοχή κλειδαριάς από το άλλο νήμα) το OS με Context Switch το μπλόκαρε και το έβαζε σε κατάσταση αναμονής μέχρι ο ιδιοκτήτης του mutex να καλέσει την unlock. Όταν αυτό γινόταν διαθέσιμο, γινόταν μια δεύτερη εναλλαγή περιβάλλοντος για την αποκατάσταση του νήματος και αυτό πλέον εκτελούσε την εντολή ατομικά και αδιαίρετα. Μετά, όταν έβγαине ξεκλείδωνε το mutex. Οπότε, σημειώνουμε πως το κλείδωμα mutex, εμπεριέχει 2 εναλλαγές περιβάλλοντος, εμπλέκοντας άμεσα το λειτουργικό σύστημα.

Παρατηρούμε ,αρχικά, ότι ο χρόνος χρήστη (user 5.058s) υπερβαίνει τον συνολικό πραγματικό χρόνο ολοκλήρωσης (real 4.199s). Αυτό αποδεικνύει ότι ο χρονοδρομολογητής του Λειτουργικού Συστήματος τοποθέτησε τα δύο νήματα σε διαφορετικούς πυρήνες, επιτρέποντας την ταυτόχρονη εκτέλεσή τους.

Σε πλήρη αντίθεση με τον ασυγχρόνιστο κώδικα, εδώ ο επεξεργαστής έκανε 2.952s εντός του Kernel Space. Αυτό οφείλεται, στο γεγονός πως τα κλειδώματα mutexes, όπως εξηγήσαμε παραπάνω εμπλέκουν άμεσα το OS, και προκαλούν μεγάλες χρονικές επιβαρύνσεις λόγω των εναλλαγών περιβάλλοντος.

Ο συνολικός χρόνος real (4.199s) είναι τάξεις μεγέθους μεγαλύτερος από την ασυγχρόνιστη έκδοση, όπως αναμέναμε λόγω των παραπάνω.

- Συγχρονισμένο simplesync.c με gcc atomic operations (εκτελέσιμο simplesync-atomic):

```
[oslab018@os-node2:~/sync$ time ./simplesync-atomic
About to increase variable 10000000 times
About to decrease variable 10000000 times
Done increasing variable.
Done decreasing variable.
OK, val = 0.

real    0m1.897s
user    0m3.768s
sys     0m0.004s
```

Το αποτέλεσμα του *val* είναι σωστό και ντετερμινιστικό σε κάθε εκτέλεση. Παρατηρούμε ότι ο μετρητής παίρνει πάντα τη τιμή μηδέν που είναι και η ορθή. Αυτό επιβεβαιώνει ότι οι ενσωματωμένες ατομικές συναρτήσεις του μεταγλωττιστή GCC (*__sync_add_and_fetch* και *__sync_sub_and_fetch*) συγχρόνισαν επιτυχώς τα νήματα.

Αρχικά, βλέπουμε ξανά ότι το Λειτουργικό Σύστημα χρονοδρομολόγησε τα νήματα ταυτόχρονα σε δύο διαφορετικούς πυρήνες, αφού *user > real time*.

System Time: Σε πλήρη αντίθεση με την έκδοση των *Mutexes* στο ίδιο Linux σύστημα (όπου ο χρόνος *sys* ήταν τεράστιος λόγω *context switches*), εδώ ο χρόνος στον *Kernel Space* είναι πρακτικά μηδενικός (0.004s). Αυτό συμβαίνει διότι οι ατομικές λειτουργίες υλοποιούνται απευθείας στο υλικό μέσω ειδικών εντολών της *Assembly*.

Real Time 1.897s: Ο συνολικός χρόνος εκτέλεσης είναι εμφανώς μικρότερος από αυτόν των *Mutexes*, καθώς γλιτώσαμε το βαρύ κόστος των εναλλαγών περιβάλλοντος. Ωστόσο, είναι πιο αργός από τον εντελώς ασυγχρόνιστο κώδικα.

Ερώτημα 2

Η χρήση **ατομικών λειτουργιών** είναι στην άσκηση μας πολύ πιο γρήγορη και αποδοτική από τη χρήση **POSIX mutexes**, καθώς στο συγκεκριμένο σενάριο απαιτείται απλά η ενημέρωση μιας κοινόχρηστης μεταβλητής, όπως η αποτελεί η αύξηση ενός μετρητή.

Πιο συγκεκριμένα, όπως γνωρίζουμε για τα **POSIX mutexes** εάν ένα νήμα προσπαθήσει να αποκτήσει ένα mutex το οποίο δεν είναι διαθέσιμο, το νήμα μπλοκάρεται και τίθεται σε κατάσταση αναμονής από το Λειτουργικό Σύστημα. Η συγκεκριμένη αναστολή λειτουργίας προκαλεί εναλλαγή περιβάλλοντος όπου απαιτεί σημαντικό χρόνο και συνιστά καθαρά χαμένο χρόνο για τη CPU, αφού κατά τη διάρκειά της δεν παράγεται κανένα απολύτως ωφέλιμο έργο. Επίσης, όπως αναφέρει και το βιβλίο, τα κλειδώματα αμοιβαίου αποκλεισμού αποτελούν μια απαισιόδοξη στρατηγική συγχρονισμού, καθώς υποθέτουν προκαταβολικά ότι θα υπάρξει ταυτόχρονη πρόσβαση από άλλα νήματα και κατά συνέπεια δεσμεύουν την κλειδαριά πριν επιτρέψουν οποιαδήποτε διάβασμα/ενημέρωση στα δεδομένα.

Αντίθετα, γνωρίζουμε πως οι **ατομικές λειτουργίες** είναι θεμελιωμένες σε μια

αισιόδοξη προσέγγιση και υλοποιούνται απευθείας στο χαμηλότερο επίπεδο του υλικού. Επειδή οι μαθηματικές και λογικές πράξεις με ατομικούς ακεραίους εκτελούνται εξ ολοκλήρου στον χώρο του χρήστη χωρίς καμία απολύτως διακοπή, δεν απαιτούν ποτέ τη χρονοβόρα επιβάρυνση των μηχανισμών κλειδώματος.

Το αποτέλεσμα, όπως αναδείχθηκε και πειραματικά στο προηγούμενο ερώτημα, είναι πως με την χρήση ατομικών εντολών πετυχαίνουμε πλήρη εξάλειψη των εναλλαγών περιβάλλοντος λειτουργίας, επιτρέποντας στον κώδικα να τρέχει πιο γρήγορα σε καταστάσεις όπου δεν υπάρχει ανταγωνισμός ή υπάρχει μέτριος ανταγωνισμός, όπως στη συγκεκριμένη άσκηση.

Ωστόσο, γενικότερα και πέρα από το ζητούμενο της άσκησης αυτής, σημειώνουμε πως σε προβλήματα υψηλού ανταγωνισμού υποβαθμίζεται η απόδοση τους και προτιμούνται τα Posix mutexes. Επίσης, η χρήση των gcc atomic operations περιορίζεται σε σενάρια απλών, μοναδικών πράξεων. Επομένως για καταστάσεις όπου υπάρχουν αρκετές μεταβλητές οι οποίες συνεισφέρουν σε μια πιθανή κατάσταση συναγωνισμού πρέπει να χρησιμοποιηθούν τα mutexes.

Ερώτημα 3

Οι παρακάτω απαντήσεις είναι βασισμένοι στην αρχιτεκτονική του **orion** της σχολής:

```
[oslab018@os-node2:~/sync$ uname -m  
x86_64
```

Η αρχιτεκτονική για την οποία μεταγλωττίζουμε είναι η x86_64 (Intel/AMD), όπως επιβεβαιώνεται από τη χρήση των 64-bit καταχωρητών (π.χ. %rbx) και του προθέματος lock στον παραγόμενο κώδικα Assembly:

Από την έξοδο της εντολής make για τον τρόπο μεταγλώττισης του simplesync.c. προς το εκτελέσιμο simplesync-atomic γράφουμε την εξής εντολή στο τερματικό:

```
gcc -Wall -O2 -g -pthread -DSYNC_ATOMIC -S simplesync.c -o  
simplesync-atomic.s
```

Για την __sync_add_and_fetch παρατηρούμε ότι η αντίστοιχη μετάφραση της

σε κώδικα Assembly είναι η :

```
.L2:
    .loc 1 47 3 is_stmt 1 view .LVU10
    .loc 1 50 4 view .LVU11
    lock addl    $1, (%rbx)
    .loc 1 46 21 view .LVU12
.LVL5:
```

Το παραπάνω τμήμα κώδικα, βρίσκεται μέσα στην ταμπέλα *increase_fn* όπου είναι και το όνομα της συνάρτησης που εκτελεί το νήμα που αυξάνει τον μετρητή(*ip). Παρατηρούμε, πως ουσιαστικά είναι η εντολή **lock addl \$1, (%rbx)** η οποία ουσιαστικά αποτελεί τον τρόπο υλοποίησης της ατομικής μας εντολής. Επιβεβαιώνουμε έτσι, ότι η *__sync_add_and_fetch* είναι μια ατομική εντολή hardware, η οποία μεταφράζεται σε εντολή μηχανής με το πρόθεμα **lock**.

Ομοίως, για την *__sync_sub_and_fetch*:

```
.p2align 3
.L7:
    .loc 1 81 3 is_stmt 1 view .LVU30
    .loc 1 84 4 view .LVU31
    lock subl    $1, (%rbx)
    .loc 1 80 21 view .LVU32
.LVL14:
```

Το παραπάνω τμήμα κώδικα, βρίσκεται μέσα στην ταμπέλα *decrease_fn* όπου είναι και το όνομα της συνάρτησης που εκτελεί το νήμα που μειώνει τον μετρητή(*ip). Παρατηρούμε, πως ουσιαστικά είναι η εντολή **lock subl \$1, (%rbx)** η οποία αποτελεί τον τρόπο υλοποίησης της ατομικής μας εντολής. Επιβεβαιώνουμε έτσι, ότι η *__sync_sub_and_fetch* είναι επίσης μια ατομική

εντολή hardware, η οποία μεταφράζεται σε εντολή μηχανής με το πρόθεμα **lock**.

Από τις παραπάνω 2 μεταφράσεις, μπορούμε να παρατηρήσουμε ότι η διεύθυνση της μεταβλητής `ip` βρίσκεται στον καταχωρητή `%rbx`.

Τώρα θα δοκιμάσουμε να τρέξουμε και να μετταγλωτίσουμε τον κώδικα στην αρχιτεκτονική **ARM64 (Apple Silicon M4)** του τοπικού μας συστήματος.

```
● dionysiskatsetis@MAC-61000D lab2_ans % uname -m  
arm64
```

Από εξωτερικές πηγές ψάξαμε και βρήκαμε πως στη συγκεκριμένη αρχιτεκτονική RISC, οι ατομικές λειτουργίες του GCC δεν μεταφράζονται πλέον στο πρόθεμα `lock` της x86, αλλά σε ένα διαφορετικό μοντέλο συγχρονισμού που βασίζεται σε εντολές αποκλειστικής προσπέλασης μνήμης, όπως θα επιβεβαιώσουμε και παρακάτω:

Για την `__sync_add_and_fetch` παρατηρούμε ότι η αντίστοιχη μετάφραση της σε κώδικα Assembly είναι η :

```
Ltmp2:  
    ;DEBUG_VALUE: increase_fn:i <- 0  
    .loc      0 0 2  
    mov w8, #1  
Ltmp3:  
LBB0_1:  
    ;DEBUG_VALUE: increase_fn:arg <- $x19  
    ;DEBUG_VALUE: increase_fn:i <- [DW_OP_  
    .loc      0 50 4 is_stmt 1  
    ldaddal w8, w9, [x19]  
Ltmp4:
```


Για την `__sync_sub_and_fetch` παρατηρούμε ότι η αντίστοιχη μετάφραση της σε κώδικα Assembly είναι η :

```
mov w8, #-1
Ltmp13:
LBB1_1:
;DEBUG_VALUE: decrease_fn:arg <- $x19
;DEBUG_VALUE: decrease_fn:i <- [DW_OP
.loc 0 84 4 is_stmt 1
ldaddal w8, w9, [x19]
```

Στον **ARM64**, παρατηρούμε ότι τόσο η ατομική αύξηση όσο και η ατομική μείωση υλοποιούνται με την εντολή *ldaddal*. Η διαφοροποίηση επιτυγχάνεται μέσω του περιεχομένου του καταχωρητή πηγής **w8**, ο οποίος στην περίπτωση της `increase_fn` περιέχει το +1 ενώ της `decrease_fn` το -1.

Ερώτημα 4

Η αρχιτεκτονική για την οποία μεταγλωττίζουμε είναι η **x86_64** (Intel/AMD) του **orion**:

Από την έξοδο της εντολής `make` για τον τρόπο μεταγλώττισης του `simplesync.c` προς το εκτελέσιμο `simplesync-mutex` γράφουμε την εξής εντολή στο τερματικό:

```
gcc -Wall -O2 -g -pthread -DSYNC_MUTEX -S simplesync.c -o  
simplesync-mutex.s
```

Για την `pthread_mutex_lock()` βρήκαμε την εξής μετάφραση σε Assembly:

```
.loc 1 47 3 view .LVU15
.LBB3:
.loc 1 54 4 view .LVU16
.loc 1 54 14 is_stmt 0 view .LVU17
movq    %r13, %rdi
call    pthread_mutex_lock@PLT
.LVL4:
movl    %eax, %ebx
```

Η μετάφραση σε Assembly αποκαλύπτει ότι ο συγχρονισμός δεν υλοποιείται από μία μεμονωμένη εντολή του επεξεργαστή, αλλά μέσω μιας κλήσης βιβλιοθήκης.

Συγκεκριμένα, όπως φαίνεται στο στιγμιότυπο από τη συνάρτηση **increase_fn**, η μεταγλώττιση παράγει την εντολή: **call pthread_mutex_lock@PLT** αφού πρώτα έχει φορτωθεί η διεύθυνση του mutex στον καταχωρητή **%rdi** μέσω της **movq %r13, %rdi**.

Η χρήση της εντολής call εξηγεί τα πειραματικά μας αποτελέσματα τα οποία ανέδειξαν υψηλό system time και context switches. Πιο συγκεκριμένα, σε αντίθεση με τις ατομικές εντολές hardware, η **call** αποτελεί μια εντολή κλήσης ρουτίνας σε μια εξωτερική βιβλιοθήκη, αναγκάζοντας έτσι το πρόγραμμα να διακόψει τη ροή του (ουσιαστικά κάνει jump σε μια διεύθυνση and link για να αποθηκεύσει στη στοίβα το return address). Εάν το mutex είναι κατειλημμένο, η συνάρτηση αυτή θα εκτελέσει μια κλήση συστήματος (**futex**) προς τον πυρήνα, αναγκάζοντας το Λειτουργικό Σύστημα να παρέμβει για να μπλοκάρει το νήμα και να το τοποθετήσει σε ουρά αναμονής, διαδικασία που επιφέρει σημαντική χρονική επιβάρυνση.

Μέρος 2

Αρχικά, φτιάξαμε την βιβλιοθήκη utils.{c,h} η οποία μας βοήθησε να δομήσουμε το πρόγραμμα μας και να ορίσουμε χρήσιμες συναρτήσεις και structs:

utils.c:

```
#include "utils.h"

#include <stdio.h>

#include <unistd.h>

#include <pthread.h>

#include <stdlib.h>

void check(int res, const char* msg){

    if(res==1){
```

```

    perror(msg);

    _exit(0);

}

}

int safe_atoi(char *s, int *val)
{
    long l;

    char *endp;

    l = strtol(s, &endp, 10);

    if (s != endp && *endp == '\0') { // check if at least one digit was read and if the whole string converted to a
number

        *val = l;

        return 0;

    } else

        return -1;

}

void *safe_malloc(size_t size)
{
    void *p;

    if ((p = malloc(size)) == NULL) {

        fprintf(stderr, "Out of memory, failed to allocate %zd bytes\n",
            size);

        exit(1);

    }

    return p;

}

void usage(char *argv0)
{
    fprintf(stderr, "Usage: %s thread_count \n\n"

```

```

    "Exactly 1 argument required:\n"

    "  thread_count: The number of threads to create.\n",

    argv0);

    _exit(1);

}

```

utils.h:

```

#pragma once

#include <stdlib.h>

#include <pthread.h>

void check(int res, const char* msg);

int safe_atoi(char *s, int *val);

void *safe_malloc(size_t size);

typedef struct {

    pthread_t tid; /* POSIX thread id, as returned by the library */

    int thrId; /* Application-defined thread id */

    int thrcnt;

} thread_i;

void usage(char *argv0);

```

Πιο συγκεκριμένα, η βιβλιοθήκη περιλαμβάνει τα εξής:

1. Δομή Δεδομένων Νημάτων (struct thread_i) Επειδή η συνάρτηση pthread_create επιτρέπει το πέρασμα μόνο ενός ορίσματος (τύπου void *) στη συνάρτηση του νήματος (worker), δημιουργήσαμε τη δομή thread_i.

- tid: Αποθηκεύει το πραγματικό POSIX thread ID που επιστρέφει το

σύστημα.

- `thrid`: Το λογικό ID του νήματος που δίνουμε στο πρόγραμμα.
- `thrent`: Το συνολικό πλήθος των νημάτων που συμμετέχουν στον υπολογισμό = `NTHREAD`

2. Ασφαλής Διαχείριση Μνήμης (`safe_malloc`): Υπήρχε στον δοθέν κώδικα `pthread-test.c`. Η συνάρτηση `malloc` δεν εγγυάται την επιτυχή δέσμευση μνήμης. Σε περίπτωση αποτυχίας, τυπώνει ένα κατατοπιστικό μήνυμα λάθους στο `stderr` (αναφέροντας τα bytes που ζητήθηκαν) και τερματίζει το πρόγραμμα.

3. Ασφαλής Μετατροπή Εισόδου (`safe_atoi`) : Υπήρχε στον δοθέν κώδικα `pthread-test.c`. Χρησιμοποιεί τη συνάρτηση `strtol`. Ελέγχει μέσω του δείκτη `endp` αν η είσοδος μετατράπηκε πλήρως και σωστά σε ακέραιο.

4. Κεντρική Διαχείριση Σφαλμάτων (`check`) Αρκετές κλήσεις συστήματος και συναρτήσεις βιβλιοθήκης (όπως αυτές των σηματοφόρων) επιστρέφουν -1 σε περίπτωση σφάλματος. Η συνάρτηση `check` δέχεται το αποτέλεσμα της κλήσης και ένα μήνυμα. Αν εντοπίσει σφάλμα, καλεί την `printf()` για να τυπώσει την ακριβή αιτία που κατέγραψε το Λειτουργικό Σύστημα και τερματίζει ομαλά την εκτέλεση με `_exit()`.

5. Καθοδήγηση Χρήστη (`usage`) Σε περίπτωση που ο χρήστης εκτελέσει το πρόγραμμα με λάθος αριθμό ορισμάτων (π.χ. παραλείποντας τον αριθμό των νημάτων), η `usage` τυπώνει τις σωστές οδηγίες χρήσης στο `stderr` και τερματίζει το πρόγραμμα.

Επιπλέον, με έμπνευση από το πρώτο μέρος, επεκτείναμε τον κώδικα που μας δόθηκε και φτιάξαμε 2 εκτελέσιμα από ένα αρχείο πηγαίου κώδικα (**`mandel.c`**), τα οποία με την ίδια λογική του πρώτου μέρους τα ονομάσαμε **`mandel-sem`** για την έκδοση με τους σηματοφόρους και **`mandel-cv`** για την έκδοση με τις μεταβλητές περιβάλλοντος.

Το **Makefile** τροποποιήθηκε κατάλληλα:

```
## Mandel sync (two versions)
```

```
mandel-sem: mandel-lib.o mandel-sem.o utils.o
```

```
$(CC) $(CFLAGS) -o mandel-sem mandel-lib.o mandel-sem.o utils.o $(LIBS)
```

```
mandel-cv: mandel-lib.o mandel-cv.o utils.o
```

```
$(CC) $(CFLAGS) -o mandel-cv mandel-lib.o mandel-cv.o utils.o $(LIBS)
```

mandel-lib.o: mandel-lib.h mandel-lib.c

`$(CC) $(CFLAGS) -c -o mandel-lib.o mandel-lib.c`

mandel-sem.o: mandel.c utils.h

`$(CC) $(CFLAGS) -DSYNC_SEM -c -o mandel-sem.o mandel.c`

mandel-cv.o: mandel.c utils.h

`$(CC) $(CFLAGS) -DSYNC_CV -c -o mandel-cv.o mandel.c`

utils.o: utils.c utils.h

`$(CC) $(CFLAGS) -c utils.c -o utils.o`

clean:

`rm -f *.s *.o pthread-test simplesync-atomic simplesync-mutex mandel-sem mandel-cv`

Ερώτημα 1

Για το σχήμα συγχρονισμού που υλοποιήσαμε χρειάστηκαν όσοι **σημοφόροι** όσα τα *NTHREADS* που δίνονται στη γραμμή εντολών από των χρήστη. Πιο συγκεκριμένα φτιάξαμε (δυναμικά) έναν πίνακα με *NTHREADS* θέσεις όπου τα στοιχεία του είναι τύπου *sem_t*, *sem_t local_lock[NTHREADS]*. Αυτό επιλέχθηκε για τους εξείς λόγους:

- Η εκτύπωση στο τερματικό δεν αρκεί να γίνεται απλώς με έναν δυαδικό σημαφόρο που ουσιαστικά εξασφαλίζει αμοιβαίο αποκλεισμό (δηλαδή απλά να μην τυπώνουν δύο νήματα ταυτόχρονα). Απαιτείται αυστηρή σειριακή σειρά (Γραμμή 0, Γραμμή 1, Γραμμή 2 κ.ο.κ.) για να μην καταστραφεί η ορθή εικόνα του Mandelbrot. Κάθε νήμα μπορεί να τελειώσει τον υπολογισμό της γραμμής του σε διαφορετικό χρόνο (οι γραμμές στο κέντρο του σχήματος αργούν πολύ περισσότερο από αυτές στα άκρα).

Αν χρησιμοποιούσαμε μόνο 1 σημαφόρο, θα λειτουργούσε σαν απλό mutex. Αυτό θα είχε ως αποτέλεσμα τα πιο γρήγορα νήματα να τυπώσουν τις γραμμές τους πριν τα πιο αργά, ανακατεύοντας την εικόνα.

- Πώς λειτουργούν οι NTHREADS σημαφόροι: Κατά την αρχικοποίηση, ο σημαφόρος του πρώτου νήματος (*local_lock[0]*) αρχικοποιείται στο 1,

ώστε να τυπώσει πρώτο, ενώ όλοι οι υπόλοιποι στο 0, ώστε να μπλοκάρουν αν προσπαθήσουν να τυπώσουν. Όταν το νήμα i τελειώσει τον υπολογισμό, κάνει `wait` στον δικό του σημαφόρο. Αν δεν είναι η σειρά του, περιμένει. Μόλις τυπώσει, δίνει το “τοπικό κλειδί” στο επόμενο νήμα κάνοντας `signal (sem_post)` στον σημαφόρο `local_lock[(i + 1) % NTHREADS]`, ξυπνώντας το στοχευμένα. Αυτό γίνεται κυκλικά και έτσι εξασφαλίζουμε την ορθή τύπωση του αποτελέσματος.

Ερώτημα 2

Τρέξαμε την εντολή `cat /proc/cpuinfo` και βρήκαμε ότι οι φυσικοί πυρήνες του `orion` είναι 8. Τα παράλληλα προγράμματα τα τρέξαμε με **2 νήματα**-εργάτες.

Για το σειριακό πρόγραμμα **mandel** απαιτήθηκε:

```
real      0m1.696s
user      0m1.651s
sys       0m0.045s
```

Για το παράλληλο πρόγραμμα **mandel-sem** απαιτήθηκε:

```
real      0m0.717s
user      0m1.250s
sys       0m0.029s
```

Για το παράλληλο πρόγραμμα **mandel-cv** απαιτήθηκε:

```
real      0m0.539s
user      0m0.921s
sys       0m0.057s
```

Παρατηρήσεις:

- Στο σειριακό πρόγραμμα, ο πραγματικός χρόνος (1.696s) είναι σχεδόν ταυτόσημος με τον χρόνο επεξεργαστή (1.651s). Αυτό είναι απολύτως αναμενόμενο, καθώς μια μονονηματική διεργασία απασχολεί αποκλειστικά έναν πυρήνα.
- Στα παράλληλα προγράμματα, παρατηρούμε το φαινόμενο ο χρόνος user να είναι μεγαλύτερος από τον χρόνο real (e.g., στο mandel-sem: user 1.250s > real 0.717s). Αυτό επιβεβαιώνει την παράλληλη εκτέλεση όπου ο χρόνος user αθροίζει τον χρόνο που κατανάλωσε η CPU σε όλους τους πυρήνες που δούλευαν ταυτόχρονα.

Σύγκριση Συγχρονισμού: Semaphores vs. Condition Variables:

- Από τις μετρήσεις προκύπτει μια ξεκάθαρη υπεροχή της υλοποίησης με τις Μεταβλητές Συνθήκης (mandel-cv), η οποία ολοκληρώθηκε σε 0.539s έναντι 0.717s της υλοποίησης με σηματοφόρους:

Η υλοποίηση με τον πίνακα από Μεταβλητές Συνθήκης που επιλέξαμε αποδείχθηκε εξαιρετικά αποδοτική. Με το μοντέλο Signal and Continue που ισχύει (δηλαδή το νήμα που ξυπνά ένα κοιμούμενο θα συνεχίσει να κρατά τη κλειδαριά μέχρι να αποφασίσει να τη ελευθερώσει και αυτή να αποκτηθεί από αυτό που ξύπνησε), το νήμα ξύπναγε στοχευμένα μόνο το

νήμα που είχε σειρά, ελαχιστοποιώντας τα περιττά context switches.

Ερώτημα 3

Στη δεύτερη εκδοχή του προγράμματός μας όπου παράξαμε το εκτελέσιμο **mandel-cv** δεν χρησιμοποιήσαμε μόνο μία μεταβλητή συνθήκης, αλλά έναν πίνακα από μεταβλητές συνθήκης (*pthread_cond_t out[NTHREADS]*), όπου κάθε κελί του πίνακα αντιστοιχεί και σε ένα ξεχωριστό νήμα.

Αν είχαμε επιλέξει να χρησιμοποιήσουμε μόνο μία, κοινή μεταβλητή συνθήκης, η λογική του συγχρονισμού θα άλλαζε ριζικά. Πιο συγκεκριμένα, όλα τα νήματα που περιμένουν τη σειρά τους θα κοιμούνταν πάνω σε αυτή τη μοναδική μεταβλητή, καλώντας την *pthread_cond_wait*. Όταν το νήμα που μόλις εκτύπωσε τη γραμμή του έπρεπε να δώσει τη σειρά στο επόμενο, **δεν** θα μπορούσε να χρησιμοποιήσει την *pthread_cond_signal*. Η συνάρτηση αυτή θα ξυπνούσε ένα τυχαίο νήμα από αυτά που περιμένουν, το οποίο πιθανότατα δεν θα ήταν αυτό που έχει σειρά. Επομένως, θα αναγκαζόμασταν να χρησιμοποιήσουμε την **pthread_cond_broadcast**, στέλνοντας σήμα σε όλα ανεξαιρέτως τα νήματα που κοιμούνται.

Thundering Herd Problem: Η χρήση της *pthread_cond_broadcast* σε προβλήματα αυστηρής σειράς, όπως το δικό μας δημιουργεί ένα σοβαρό πρόβλημα επίδοσης:

- Το νήμα κάνει broadcast και ξεκλειδώνει τη κλειδαριά.
- Όλα τα υπόλοιπα νήματα ξυπνούν ταυτόχρονα.
- Όλα μαζί ανταγωνίζονται για να δεσμεύσουν το κλειδί.
- Το OS το δίνει σε ένα νήμα. Αυτό το νήμα ελέγχει τη συνθήκη *while(turn != thrid)*. Επειδή κατά πάσα πιθανότητα δεν είναι η σειρά του, αφήνει το κλειδί και ξαναπάει για ύπνο.
- Το κλειδί περνάει στο επόμενο νήμα, το οποίο κάνει το ίδιο, μέχρι να το πάρει τελικά το ένα νήμα που έχει όντως σειρά.

Αυτή η διαδικασία αναγκάζει το Λειτουργικό Σύστημα να κάνει συνεχόμενα και άσκοπα context switches που σπαταλούν τεράστιο ποσοστό πόρων της CPU και αυξάνουν κατακόρυφα τον χρόνο συστήματος.

Ερώτημα 4

Ναι, το παράλληλο πρόγραμμα εμφανίζει σημαντική επιτάχυνση σε σχέση με το σειριακό. Όπως έδειξαν οι χρονομετρήσεις μας στον Orion (8 πυρήνες), ο πραγματικός χρόνος εκτέλεσης μειώθηκε από τα 1.696s στα 0.539s. Αυτό συμβαίνει επειδή το βαρύ υπολογιστικό τμήμα της εφαρμογής, δηλαδή η συνάρτηση `compute_mandel_line`, εκτελείται παράλληλα από πολλά νήματα σε διαφορετικούς πυρήνες της CPU.

- *Παρόλο που υπάρχει επιτάχυνση, αυτή δεν είναι γραμμική (δηλαδή με 8 πυρήνες δεν πήραμε 8 φορές ταχύτερο αποτέλεσμα). Το πρόβλημα εντοπίζεται στη σειριοποίηση της εξόδου, καθώς η εμφάνιση των γραμμών στο τερματικό πρέπει να γίνεται με αυστηρή σειρά .*

Το κρίσιμο τμήμα στην υλοποίησή μας περιέχει μόνο τη φάση εξόδου κάθε γραμμής που παράγεται:

- Είναι το ελάχιστο δυνατό. Περιέχει μόνο την εκτύπωση της γραμμής `output_mandel_line`. (και την ενημέρωση του επόμενου νήματος στη δεύτερη έκδοση `mandel-cv`).
- **Δεν χρειάζεται να περιέχει και την φάση του υπολογισμού.** Αν το κρίσιμο τμήμα περιλάμβανε και τη φάση υπολογισμού (`compute_mandel_line`), δεν θα είχαμε ουσιαστικά παραλληλισμό άρα ούτε speedup, καθώς μόνο ένα νήμα θα μπορούσε να υπολογίζει τη γραμμή του κάθε φορά. Έτσι το πρόγραμμα μας θα γινόταν σειριακό, παρά τη χρήση πολλών νημάτων.

Ερώτημα 5

Όταν πατάμε Ctrl-C ενώ το πρόγραμμα εκτελείται, το πρόγραμμα λαμβάνει το σήμα SIGINT, όπου η προκαθορισμένη του συμπεριφορά είναι να τερματίζεται βίαια. **Ως αποτέλεσμα, το τερματικό αφήνεται σε αλλοιωμένη κατάσταση.** Όπως επιβεβαιώθηκε και πειραματικά, αν το πρόγραμμα διακοπεί τη στιγμή που υπολογίζει και τυπώνει π.χ. μια **μπλε** γραμμή, το τερματικό παραμένει κλειδωμένο στο μπλε χρώμα.

